

Agentic Coding for Economists

Day 1: Foundations, GitHub, Project Brief

Participant repo: https://github.com/meleantonio/AC4E_Pavia

Antonio Mele
London School of Economics

Workshop at University of Pavia

June 22–24, 2026

What is Agentic Coding?

Definition

Agentic coding = using an AI system that can **reason over a task**, **inspect project files**, **propose a plan**, **edit when allowed**, **run checks when allowed**, and **report evidence**.

- **Beyond chat**: the agent can often interact with your project, not just answer a question.
- **Human-owned**: the research question, identification argument, data rights, interpretation, and merge decision stay with you.
- **Verifiable**: serious outputs come with diffs, logs, tests, replication checks, or review notes.

What the Agent Does Not Own

Agents can help with:

- explaining code and repository structure;
- drafting briefs, docs, and table notes;
- translating design into small tasks;
- scripts in R, Stata, MATLAB, Python, Julia, or shell;
- replication README sections and bibliography checks;
- reviewing logs, traces, and overclaimed results.

Fast research assistant, not coauthor: clear instructions, narrow context, human review.

The economist still owns:

- the research question;
- the identification argument;
- interpretation of causal evidence;
- data access rights and confidentiality;
- final claims in a paper or presentation;
- the decision to merge into main.

Why Economists Need Agentic Coding

- **Pipeline complexity:** Data loading → cleaning → estimation → tables/figures → paper. Many small, automatable steps.
- **Reproducibility pressure:** Journals and replicators expect clean code, documented data, one-command runs.
- **Time-to-insight:** AI accelerates boilerplate (documentation, tests, LaTeX tables) so you focus on design and interpretation.
- **Multi-artifact projects:** Code + paper + appendix + replication package. Agents keep them consistent.

Why Economists REALLY Need Agentic Coding

“Hi Antonio, I am just writing basically say that I owe you big time. Since the start of February I started getting into agentic coding seriously after reading your tweets. My coding skills are extremely bad and for years I have been a bit down in my research because I felt so many other economist were more tooled up than me.

In under 2 months I have managed to solve a really difficult coding problem (I have 2 computational economists co-authors that couldn't do it) and so I have nearly finished a paper in 2 weeks that has taken me about 9 years to write, nearly written another paper in under week and am burning through tokens and using my uni's supercomputer constantly. In addition I gave a workshop on agentic coding here at [...]. It is quite crazy that this has all happened in under 2 months. My research life has completely changed and am enjoying research for the first time since my PhD.”

— Colleague (shared with permission)

The Central Rule

Every important agent action needs three parts

- ① **Scope:** what the agent may read, change, or run.
- ② **Evidence:** what output, diff, log, table, or explanation shows what happened.
- ③ **Review:** what a human checks before accepting the result.

If one is missing, slow down. Most agent failures in research projects are not model failures; they are scope, evidence, or review failures.

Software and Language Lanes

Lane	Common files	Typical verification
R	.R, .Rmd, renv.lock	Rscript scripts/run_analysis.R; testthat checks
Stata	.do, .ado, .dta, logs	stata-mp -b do scripts/run_analysis.do; inspect logs
MATLAB	.m, .mlx, .mat	matlab -batch "run(...)"
Python	.py, notebooks, pyproject.toml	python scripts/run_analysis.py; pytest
Julia	.jl, Project.toml	julia -project=. ...
LaTeX	.tex, .bib, figures/tables	latexmk -pdf; citation checks
Mixed	Excel, shell, notebooks, proprietary tools	manual run checklist plus expected outputs

Prompt habit: "My main implementation lane is Stata/MATLAB/R/etc. Do not translate the project unless I ask."

Workshop example

What happened to fast-food employment in New Jersey relative to eastern Pennsylvania after New Jersey raised its minimum wage in April 1992?

- ➊ **Find and document** the public data source.
- ➋ **Identify** unit, treatment, comparison group, outcome, timing, and sample restrictions.
- ➌ **Build** a cleaned analysis file and baseline comparison.
- ➍ **Review** sensitivity, limitations, and claims.
- ➎ **Package** a replication README and final presentation.

Fork and clone this repo before we start:

AC4E_Pavia

github.com/meleantonio/AC4E_Pavia

Same public repo for the first **35–40 minutes** — then apply the pattern to your own research workflow.

Long-form workshop reference

GUIDE.md is the participant textbook. Read it before, during, and after the live sessions.

- **Full guide:** GUIDE.md
- **Day 1 chapter:** guide/01-foundations.md
- **Exercises:** exercises/day1-foundations.md
- **Day schedule:** days/day1.md

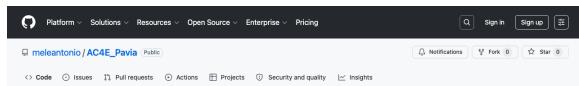
Read-only opener (from the guide):

Read START_HERE.md, GUIDE.md, AGENTS.md, and days/day1.md. Do not edit files. Tell me what to do first, which files to avoid, and which command verifies setup.

Fork the Participant Repo

- ❶ **Open upstream repo:**
`https://github.com/meleantonio/AC4E_Pavia`
- ❷ **Click Fork** (top right on the repo page).
- ❸ **Confirm your account** as the fork owner.
- ❹ **Keep the fork public** unless your project includes restricted data.

Do **not** push exercise work directly to upstream main.



Real GitHub UI — verify button labels in your browser version.

Clone Your Fork

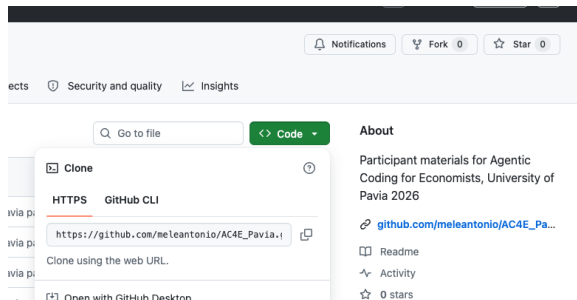
On your fork (YOUR_USER/AC4E_Pavia):

- 1 Click Code and copy the clone URL.
- 2 Clone locally (HTTPS or SSH):

```
git clone https://github.com/  
YOUR_USER/AC4E_Pavia.git  
cd AC4E_Pavia  
git status
```

SSH:

```
git@github.com:YOUR_USER/AC4E_Pavia.git
```



Clone from **your fork** (YOUR_USER/AC4E_Pavia), not
upstream meleantonio/AC4E_Pavia.

Verify and Open in Your Tool

Quick checks before we start the guided tour:

- `git status` runs and shows On branch main.
- `START_HERE.md`, `GUIDE.md`, **and** `AGENTS.md` are visible in the file tree.
- **Setup check:** `python3 -m pytest examples/mini-economics/tests`
- **Open the folder** in Cursor, VS Code, or your chosen tool (not just a single file).
- **Remote points to your fork** — run `git remote -v` if unsure.

Need help?

Raise your hand if fork, clone, or SSH keys are blocking you — we will pair for 5 minutes before moving on.

Guided Start: One Shared Project

New Day 1 opening

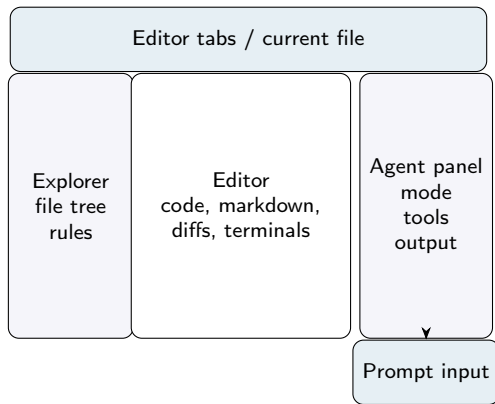
For the first **35–40 minutes**, everyone works in the same public repo: `AC4E_Pavia`. We learn the interface, run one tiny check, and complete one small git cycle **before** personal project framing starts.

- ❶ Fork and clone `AC4E_Pavia`; open it in your chosen tool.
- ❷ Use a read-only workflow to explain the file and locate the live-tour TODOs.
- ❸ Use an implementation workflow to edit one function and run `pytest`.
- ❹ Create a branch, commit, push, and open a PR.
- ❺ Then move to your own project brief and repo workflow with the same pattern.

Agent Interface at a Glance

Learn to recognize five surfaces in your tool:

- **Explorer:** files, folders, AGENTS.md, optional tool adapters
- **Editor:** current code or markdown file
- **Agent panel:** mode selector, prompt, tool output
- **Chat input:** where tasks and context instructions start
- **Source Control:** stage, commit, branch, push



Verify in your tool version: labels and button positions can change between releases.

Mode Selector and Context Entry Points

Four modes you will use today:

- **Ask**: explanation, reading, low-risk exploration
- **Agent**: edits files, runs commands, verifies
- **Plan**: breaks multi-step work into a sequence
- **Debug**: investigates failures systematically

Ways to give the agent context:

- Attach the current file or folder
- Mention exact paths in plain language
- Ask the agent to search the web when the answer is not in the repo

Workflow: **Read** **Plan** **Implement** **Debug**

Prompt input

@ current file

@ folder

"Only edit src/analysis.py"

"Search the web for ..."

Exact context controls vary by tool and release; the stable teaching idea is **scope context deliberately**.

Source Control and PR Flow

In the Source Control view, you can:

- see changed files
- stage selected edits
- write a commit message
- check your current branch
- push or sync to GitHub

Your choice: use the IDE Source Control view, terminal git, or GitHub CLI. The branch → commit → PR pattern stays the same.

Source Control

M src/analysis.py

M tests/test_analysis.py

Message: Refine guided tour summary

Stage → Commit → Push

Branch: feature/guided-tour-note

Where Governance Lives in the File Tree

```
AC4E_Pavia/  
|-- GUIDE.md  
|-- guide/  
|-- START_HERE.md  
|-- AGENTS.md  
|-- days/  
|-- exercises/  
|-- docs/  
|-- notes/  
|-- tool-lanes/  
|-- examples/  
|   '-- mini-economics/  
|-- agent-harness/  
'-- replication/
```

Key files to bookmark:

- GUIDE.md: full participant reference
- AGENTS.md: project instructions for any agent
- .cursorignore: Cursor privacy and indexing boundary
- days/day1.md: today's deliverables and flow

These files connect the interface tour to today's guardrails.

Day 1 At A Glance

Block	What is introduced	What you produce
Morning	Agent workflows, context, setup, repository orientation	tool-lane notes, setup verification
Mid-day	Governance, privacy, GitHub basics	AGENTS.md, privacy notes, first branch or PR path
Afternoon	Research question and project brief	docs/project_brief.md , initial backlog seed

Pedagogical order (from GUIDE.md): agentic coding → environment → research framing → (Days 2–3) SDD and advanced workflows → replication package.

Workshop Sequence (June 22–24)

Day 1: Foundations, GitHub, Project Brief

- ➊ **Agent workflows and setup:** read-only, planning, implementation, debugging; choose tool and language lanes.
- ➋ **Repository orientation:** open the participant repo, inspect `AGENTS.md`, verify setup.
- ➌ **Governance and GitHub:** privacy boundaries, first branch or PR path, issue vocabulary.
- ➍ **Research framing:** turn an idea into [docs/project_brief.md](#) and an initial backlog seed.

Day 2: Research Pipeline and SDD

- ➊ **Design memo:** research question, literature, hypotheses or model claims, method, limitations.
- ➋ **Data/model-input map:** document sources, primitives, parameters, equations, or solver inputs.
- ➌ **Acceptance criteria:** convert vague tasks into observable checks and prioritized issues.
- ➍ **Context patterns:** tight, wide, and external context; official docs for current tool details.

Day 3: Orchestration, Risk, Replication

- ➊ **Skills and subagents:** reusable workflows, reviewer roles, replication checks, review loops.
- ➋ **MCP, cloud agents, swarms:** add structured access and parallelism only after scope and review criteria exist.
- ➌ **Autonomous-agent risk:** complete a risk card before delegating across tools, memory, or schedules.
- ➍ **Integration:** replication README, presentation, and 30-day adoption plan.

End point: a reviewable mini research package, not a chat transcript.

How we teach

- ❶ **Short demo:** one tactical workflow, live.
- ❷ **Hands-on sprint:** you produce or revise one artifact in your own workflow.
- ❸ **Debrief:** what did the agent do, what evidence did it produce, and what did you sign off?

Today: shared guided tour first, then workflows, project guidance, privacy, GitHub, context control, and the project brief.

Day 1: Sprint 1

Agent Workflows Overview

Core workflows:

- **Read** — explain, inspect, summarize, no edits
- **Plan** — break multi-step work into a sequence
- **Implement** — edit files, run commands, verify
- **Debug** — reproduce, hypothesize, change one thing, rerun

Customization (often not a separate mode):
rules, skills, subagents, MCP.

Rule of thumb

Use read-only prompts when you want to explore or verify.

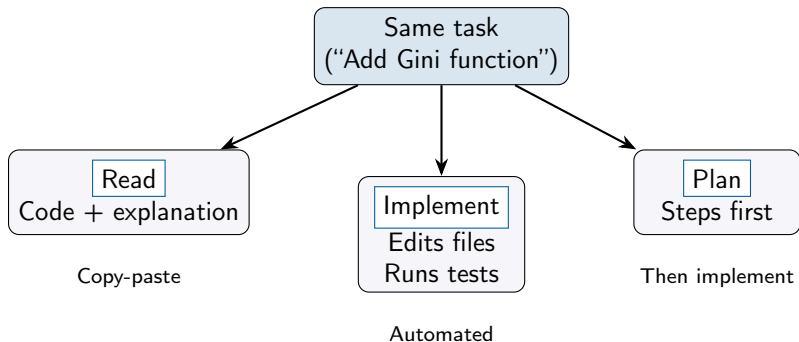
Use implementation prompts when you want files changed and tasks automated.

When to Use Each Workflow

Task	Workflow	Reason
"What's the Gini formula for grouped data?"	Read	Research, no edits
"Add Gini to src/utls/"	Implement	Code change
"Break down IV pipeline into steps"	Plan	Multi-step design
"Why does this test fail?"	Debug	Systematic diagnosis
"Refactor across 5 files"	Plan → Implement	Scope first

Economists: Use read-only prompts for literature, formula, design, or log review; implementation prompts only after scope and evidence are explicit.

Workflow Comparison Diagram



Read vs Implement: Same Task, Different Output

Read-only prompt — research/exploration:

```
# Read README.md and scripts/run_analysis.do.  
# Do not edit files. Explain inputs, outputs,  
# setup risks, and the verification command.
```

Implementation prompt — files change:

- Edits only the allowed file or issue scope.
- Runs the named command or records a manual check.
- Reports the diff, output, and remaining review points.

Both valid; choice depends on “understand” vs “change.”

Project Instructions with AGENTS.md

Shared guidance

AGENTS.md is the portable project instruction file for agents and collaborators. Keep it short enough that a human can review it and an agent can follow it.

- **Project:** what research project this repo supports.
- **Main software lane:** R, Stata, MATLAB, Python, Julia, LaTeX-heavy, or mixed.
- **Key paths:** docs, scripts, outputs, replication, private data boundaries.
- **Rules:** relative paths, no raw-data edits, no unsupported causal claims.
- **Privacy:** files and folders the agent must not read, summarize, upload, or edit.

Privacy Boundaries & Performance

What it does:

Controls what the agent may read, index, or send as context.

- **Privacy:** API keys, restricted microdata, confidential survey data, unpublished drafts.
- **Performance:** large raw files, PDFs, caches, .venv.
- **Relevance:** work from synthetic, public, or documented derived data unless told otherwise.

Different from .gitignore:

.gitignore = version control

.cursorignore = Cursor AI context

AGENTS.md = shared boundary language

Economics patterns:

```
data/raw/  
data/private/  
.env  
*.dta  
*_confidential*
```

Verify ignore behavior in your Cursor version; other tools use project instructions or settings.

Purpose

“Front door” for any agent: human-readable project instructions at repo root.

```
# Agent Instructions -- [Project Name]

## Project
[One paragraph describing the research project.]

## Main software lane
[R / Stata / MATLAB / Python / Julia / mixed.]

## Key paths
- docs/ -- research documents and notes
- scripts/ -- runnable analysis or model scripts
- outputs/ -- generated tables and figures
- replication/ -- clean-run instructions

## Rules
- Use relative paths.
- Do not edit raw data.
- Do not claim identification is valid without design evidence.

## Privacy
- Do not read .env, credentials, data/private/, or restricted files.
```

Tool Adapters Are Optional

Portable first: put durable guidance in `AGENTS.md`; use tool-specific adapters only for details that the tool can enforce or load conveniently.

- **Cursor:** rules, ignore files, MCP config, optional custom agents.
- **Claude Code:** `CLAUDE.md`, slash commands, agents, MCP config.
- **Codex:** `AGENTS.md`, skills, goals, local tools, GitHub workflows.
- **Other agents:** use whatever project instructions, memories, or settings they support.

For changing product UI, official docs beat old slides; verify field names and labels in your installed version.

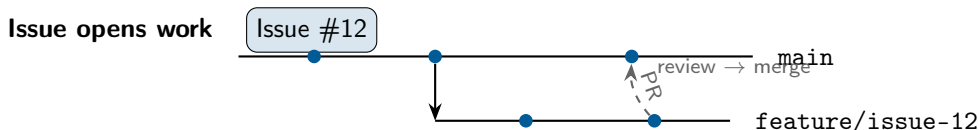
Day 1: Sprint 2

Why GitHub appears early

It gives the workshop a control plane: work is described, isolated, reviewed, and accepted outside the chat transcript.

Term	Research meaning
Repository	Folder with version history and review surface
Commit	Saved change with a message
Branch	Safe line of work separate from <code>main</code>
Issue	Task card with acceptance criteria
Pull request	Proposed change with diff and review notes
Review	Human check before accepting or merging

Issue → Branch → PR → Review → Merge



Parallel work: `main` keeps moving; your fix lives on `feature/...` until a PR merges it.

Agentic pattern: One issue per deliverable → one agent per issue → PR → consistency check before merge.

GitHub Workflow: Commands

```
git clone <repo_url>
git checkout -b feature/issue-12-literature-map
git add .
git commit -m "Add first literature mapping script"
git push -u origin feature/issue-12-literature-map
```

- `clone` — Copy the remote repo locally.
- `checkout -b` — Create and switch to a new branch for the issue.
- `add` — Stage files for the next commit.
- `commit` — Record a snapshot with a message.
- `push -u` — Publish the branch and set upstream tracking.

Without typing git: Use your IDE's **Source Control** view (branch, commit, push) and GitHub-backed features for PRs when your repo is connected.

Agent + CLI: Ask the agent to run `gh` (e.g. `gh pr create`) if GitHub CLI is configured.

Then: open or finish the PR on GitHub; request review (or run a review agent); merge after checks.

Issue Labels for Economics

Label	Use for
research-question	Framing, hypothesis refinement
literature	BibTeX, literature map
data	Sourcing, cleaning, schema
model	Theoretical setup
estimation	Code, robustness
writing	Paper, appendix
consistency-check	Review, replication gate

Day 1 goal: create a safe first issue or branch path; seed the backlog without pretending the full design is already known.

First Safe Issue

Issues before research implementation

The first issue should teach the work loop. It does not need a design memo or project brief yet.

Title: Day 1 setup verification

Acceptance criteria:

- chosen agent/tool lane recorded in `notes/context_patterns.md`;
- software lane recorded if known;
- setup check run or blocker documented;
- no private data added;
- diff reviewed before merge.

- **Title:** “Add first project brief (#12)”.
- **Description:** what changed, why, and which issue or task it satisfies.
- **Verification:** read-only review, command output, manual check, or documented blocker.
- **Research caveat:** what should *not* be inferred from this change.

Before merge: read the diff. Agent output can be polished and still be off-scope or overclaiming.

Context in the Agent panel

Pattern	Purpose	
Tight path	One memo, function, do-file, or error message	Exact context
Wide repo search	Names, paths, variable definitions, paper-table alignment	
External context	Official docs, literature, data-source metadata	
Explicit exclusions	Private data, secrets, raw files, frozen model files	

controls vary by tool and release; verify in your Cursor/Codex/Claude Code version.
Narrower context → *more accurate, less hallucinated* output.

Tight vs Wide Context

Tight context:

*“Only `src/utls/inequality.py`:
fix the empty-input edge case.”* *edit*

→ Precision, reproducibility, minimal diff.

Wide context:

*“Search the repo: add Gini across
scripts that use income data.”*

→ Consistency, integration, broader scope.

Economists: Tight for fixes/refactors; wide for cross-file coherence.

Example prompts

Examples:

- “Read README.md, START_HERE.md, GUIDE.md, AGENTS.md, and days/day1.md. Do not edit files. Explain the repository map, expected artifacts, privacy boundaries, and setup verification.”
- “List only the files I am expected to edit on Day 1 and the files I should avoid. Do not edit files.”
- “Here is my draft research idea. Do not edit files. Tell me whether it is specific enough for a three-day prototype.”

Verification: after adding boundaries, ask what files are off limits and check the answer against your instructions.

Why the Project Brief Comes After Setup

First research artifact — not the first action

The project brief should come after you know where the repo is, how to ask read-only questions, and how to protect private files.

A brief answers:

- **Question:** what are you trying to learn or build?
- **Motivation:** why does it matter?
- **Inputs:** data, model inputs, or sources.
- **Method:** empirical strategy, model, simulation, or mapping.
- **Deliverable:** what can be done in three workshop days?
- **Out of scope:** what will not be claimed or built.

Project Brief Template

```
# Project Brief

## Title
## Research question
## Motivation
## Expected contribution
## Data or model inputs
## Method
## Three-day deliverable
## Out of scope
```

Prompt sequence (from GUIDE.md):

- ➊ Read-only critique of your draft idea.
- ➋ Draft the one-page brief in chat.
- ➌ Only then: edit [docs/project_brief.md](#).

Weak vs Better Research Questions

Weak	Better
Study minimum wages.	Did fast-food employment in New Jersey change relative to eastern Pennsylvania after the 1992 minimum wage increase?
Look at monetary policy.	How do unemployment and inflation respond after a federal funds rate shock in a quarterly VAR using FRED data?
Model competition.	In a Cournot duopoly with asymmetric costs, how does a cost shock change equilibrium quantities and profits?
Day 1 field rule: labor output is setup and brief scaffold, not the DiD estimate; macro is not yet a VAR; theory is not yet a solved model.	

Running Case Brief: Card and Krueger

- **Question:** Did NJ fast-food employment change relative to eastern PA after the April 1992 minimum wage increase?
- **Contribution:** replication-oriented teaching exercise, not a new minimum wage paper.
- **Inputs:** public Card-Krueger fast-food data; restaurant-level before/after observations.
- **Method:** baseline difference-in-differences comparison.
- **Deliverable:** documented mini replication package with one baseline table and clear limitations.
- **Out of scope:** no claim that all minimum wage policies have the same effect; no unrestricted causal generalization.

Full template and prompts: `GUIDE.md` sections 12–15 and `exercises/day1-foundations.md`.

Day 1 Exercises

Canonical exercises (`exercises/day1-foundations.md`):

- ❶ **Read the repo** — read-only map, privacy boundary, setup command.
- ❷ **Context and privacy** — scope, evidence, review in `notes/context_patterns.md`.
- ❸ **Choose lanes** — agent/tool lane and research software lane.
- ❹ **Safe first branch** — edit only notes or brief placeholder; review the diff.
- ❺ **Draft the project brief** — read-only critique first, then [docs/project_brief.md](#).

Extra practice: rewrite “Improve my project” as read-only, planning, and implementation prompts with scope, evidence, and review.

Definition of Done for Day 1

- ❶ **Tool lane recorded:** chosen agent/tool and main research-language lane.
- ❷ **Setup verified:** command or manual check recorded.
- ❸ **Governance started:** `AGENTS.md` plus privacy notes or ignore rules.
- ❹ **GitHub path practiced:** first safe branch, issue, PR path, or reviewed diff.
- ❺ **Project brief drafted:** `docs/project_brief.md` with question, inputs, method, deliverable, and out-of-scope claims.

Day 1 Deliverables Checklist

- ☐ GUIDE.md bookmarked or skimmed
- ☐ AC4E_Pavia forked and cloned
- ☐ Setup check recorded
- ☐ Tool lane and software lane recorded in notes/context_patterns.md
- ☐ AGENTS.md reviewed or drafted
- ☐ Privacy notes or ignore boundaries documented
- ☐ First branch or PR path practiced
- ☐ docs/project_brief.md (v1)
- ☐ Initial backlog seed: 2–4 issues or task notes

Research Pipeline and Spec-Driven Development

- Spec-driven development: intent → requirements → design → tasks
- Research design memo: literature, hypotheses/model claims, method, limitations
- Data or model-input map: sources, primitives, parameters, solver inputs, checks
- Context control: tight, wide, and external context
- Acceptance criteria: prioritized issues with evidence and out-of-scope notes

Deliverables: [docs/research_design_memo.md](#), bibliography/reference list, data or model-input map, [notes/context_patterns.md](#), prioritized issues.